

Database-backed visualization systems

CS524: Big Data Visualization & Analytics
Fabio Miranda



urbantk.org



evl

electronic
visualization
laboratory



UNIVERSITY OF
ILLINOIS CHICAGO

Database-backed visualization systems

- Modern interactive visualization stacks are *great at authoring* (declarative specs, linked views), but they often hit a wall when you try to handle:
 - Very large data.
 - Many coordinated components.
 - Fast interaction loops.
 - Interoperability across different view toolkits.

The Effects of Interactive Latency on Exploratory Visual Analysis

- Empirical, user-centered argument for why fast, scalable, linked interaction matter.
- Liu and Heer ran a controlled study adding **+500ms delay per interaction** into an otherwise interactive system, and measured both interaction logs and think-aloud reasoning.

Key finding: +500ms caused measurable harm. Users interacted less, covered less of the dataset, and produced fewer observations / generalizations / hypotheses

Liu and Heer, 2014

2122

IEEE TRANSACTIONS ON VISUALIZATION AND COMPUTER GRAPHICS, VOL. 20, NO. 12, DECEMBER 2014

The Effects of Interactive Latency on Exploratory Visual Analysis

Zhicheng Liu and Jeffrey Heer

Abstract—To support effective exploration, it is often stated that interactive visualizations should provide rapid response times. However, the effects of interactive latency on the process and outcomes of exploratory visual analysis have not been systematically studied. We present an experiment measuring user behavior and knowledge discovery with interactive visualizations under varying latency conditions. We observe that an additional delay of 500ms incurs significant costs, decreasing user activity and data set coverage. Analyzing verbal data from think-aloud protocols, we find that increased latency reduces the rate at which users make observations, draw generalizations and generate hypotheses. Moreover, we note interaction effects in which initial exposure to higher latencies leads to subsequently reduced performance in a low-latency setting. Overall, increased latency causes users to shift exploration strategy, in turn affecting performance. We discuss how these results can inform the design of interactive analysis tools.

Index Terms—Interaction, latency, exploratory analysis, interactive visualization, scalability, user performance, verbal analysis

1 INTRODUCTION

One stated goal of interactive visualization is to enable data analysis at “rates resonant with the pace of human thought” [19, 20]. This goal entails two research directions: understanding the rate of cognitive activities in the context of visualization, and supporting these cognitive processes through appropriately designed and performant systems.

Latency is a central issue underlying these research problems. Due to the time required for query processing, data transfer, and rendering, data-intensive visualization systems incur delay. It is generally held that low latency leads to improved usability and better user experience. Unsurprisingly, multiple research efforts focus on reducing query and rendering latency for large datasets, which may include billions or more data points. Latencies in state-of-the-art systems can range from 20 milliseconds up to multiple seconds for a unit task [2, 28, 29].

Despite the shared goal of minimizing latency, the effects of interaction delays on user behavior and knowledge discovery with visualizations remain largely unevaluated. While previous research on the effects of interactive latency in puzzle solving [4, 17, 35, 36] and search [8] has shown that user behavior changes in response to millisecond-scale differences in latency, studies in other domains such as computer games report no significant effects [23, 39].

It is unclear to what degree these findings apply to exploratory visual analysis. Unlike problem-solving tasks or most computer games, exploratory visual analysis is open-ended and does not have a clear goal state. User interaction may be triggered by salient visual cues in the display, driven by *a priori* hypotheses, or carried out through exploratory browsing. The process is more spontaneous and is unconstrained by factors such as game rules.

How does latency affect user behavior and knowledge discovery in exploratory visual analysis? To answer this question, we conduct controlled experiments comparing two latency conditions: baseline 500ms per operation. We analyze data collected from both system logs and think-aloud protocols to test if (a) delay impacts interaction strategies and (b) lower latency leads to better analysis performance.

Our work makes the following contributions. First, we present the design and the results of a controlled study confirming that a 500ms difference can have significant impacts on visual analysis. Specifically, we find that (1) the additional delay results in reduced interaction and reduced dataset coverage during analysis; (2) the rate at which users make observations, draw generalizations and generate hypotheses (as determined using a think-aloud protocol) also declines

due to the delay; and (3) initial exposure to delays can negatively impact overall performance even when the delay is removed in a later session. Second, we extend the insight-based evaluation methodology [37, 38] for comparative analysis of qualitative data regarding visualization use. We introduce a procedure for segmenting, coding and analyzing think-aloud protocols for visualization research. Our analysis contributes coding categories that are potentially applicable for future protocol analysis. Finally, our results show that the same delay has varying influences on different interactive operations. We discuss some implications of these findings for system design.

2 RELATED WORK

Our research draws on related work in scalable visualization systems, cognitive science and domain-specific investigations on the effects of interactive latency. We review relevant literature below.

2.1 Scalable Data Analysis Systems

Building low latency analysis systems has been a focus for many research projects and commercial systems, spanning both back-end and front-end engineering efforts. Spark [44, 45] supports fast in-memory cluster computing through read-only distributed datasets for machine learning tasks and interactive ad-hoc queries. Nanocubes [28] contributes a method to store and query multi-dimensional aggregated data at multiple levels of resolution in memory for visualization. Profiler [26] builds in-memory data cubes for query processing. Tableau’s data engine [1] optimizes both in-memory stores and live connections to databases on disk. imMens [29] decomposes multi-dimensional data cubes into binned data tiles of reduced dimensionality and performs accelerated query processing and rendering on the GPU.

In cases where long-running queries are unavoidable, sampling and online aggregation [22] are often used to improve user experience. BlinkDB [2] builds multi-dimensional, multi-resolution samples and dynamically estimates a query’s response time and error. With online aggregation [22], visualizations of estimated results are incrementally updated as a query progresses. Studies suggest that data analysts can interpret approximate results visualized as bar charts with error bars to make confident decisions [16].

2.2 Time Scales of Human Cognition

Decades of psychology research have produced evidence that different thought processes operate at varying speeds [25]. Newell [33] provides a framework outlining proposed time scales of human cognition. Relevant to studies of human-computer interaction are the cognitive (100 milliseconds to 10 seconds) and rational (minutes to hours) time bands. Within the cognitive band, Newell identifies three types of time constants: deliberate act, operation, and unit task. Card et al. [11] make similar distinctions using a different terminology. Table 1 summarizes these scales, exemplary actions, and the time ranges during which these actions occur.

• Zhicheng Liu is with Adobe Research. E-mail: liu@adobe.com.
Jeffrey Heer is with the University of Washington. E-mail: jheer@uw.edu.

Manuscript received 31 Mar. 2014; accepted 1 Aug. 2014. Date of publication 11 Aug. 2014; date of current version 9 Nov. 2014.
For information on obtaining reprints of this article, please send e-mail to: rvcg@computer.org.
Digital Object Identifier 10.1109/TVCG.2014.2346452

Database-backed visualization systems

- Liu and Heer results imply that system performance is a first-class cognitive constraint. In other words: **if interactive updates drift into “half-a-second territory,” exploration quality drops.**
- Works in visualization are essentially a response to that:
 - Make interactive views so common interactions stay fast through optimization (e.g., caching, indexing).
 - immens
 - Nanocubes
 - TopKube
 - Time-Lattice
 - NeuralCubes
 - ...



Database-lacked visualization systems

1036 IEEE TRANSACTIONS ON VISUALIZATION AND COMPUTER GRAPHICS, VOL. 19, NO. 12, DECEMBER 2013

Nanocubes for Real-Time Exploration of Spatiotemporal Datasets

Lauro Lins, James T. Klosowski, and Carlos Scheidegger

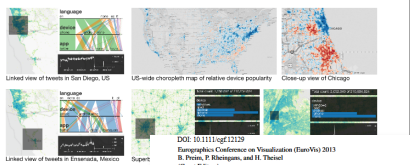


Fig. 1. Example visualizations of 210 million past purpose enables real-time (these images show us spatiotemporal, multidimensional datasets. The view rendered by a traditional linear scan over the data maps, time series over arbitrary regions and scale scale of the choropleth map is a changing as corresponds to higher relative possibility of finding.

Abstract—Consider real-time exploration of large t location, x , and y attributes. An earlier data? Answering these questions requires algorithms datasets implement low-known data cube query over the database. Data cubes are sometimes stored. In contrast, we show how to construct a d we call the data structure λ nanocube. We present λ generally we require visual knowledge such as visualizations created by scanning an entire dataset is a hierarchical view of space and time. We λ and present memory, living, and network bandwidth determined by network and user interaction interface.

Index Terms—Data cube, Data structure, Interact

1. INTRODUCTION

As datasets grow larger, exploratory data visualization becomes difficult. Consider a dataset with a billion entries. We λ a small summary of the dataset and visualize the summary the dataset, but in Amoretti's famous quote shows [1] themselves cannot answer their own queries. Summarize but in order to understand if that is the case, we will in

Lauro Lins is with AT&T Research, E-mail: lauro@research.att.com. James T. Klosowski is with AT&T Research, E-mail: jklosowski@att.com. Carlos Scheidegger is with AT&T Research, E-mail: cscheidegger@att.com. Manuscript received 11 March 2013; accepted 1 August 2013; 11 October 2013; revised 4 October 2013. For information on obtaining reprints of this article, please visit http://www.ieee.org/publications_standards/publications/rights/index.html.

Authorized licensed use limited to: University of Illinois at Chicago. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Lins et al., 2013

imMens: Real-time Visual Querying of Big Data

Zhicheng Liu*, Wayne Jiang[†] and Jeffrey Heo[‡]

^{*}Department of Computer Science, Stanford University
[†]Department of Computer Science and Technology, Tsinghua University

Abstract

Data analysts must make sense of increasingly large data sets, sometimes with billions or more records. We present methods for interactive visualization of big data, following the principle that perceptual and interactive scalability should be limited by the chosen resolution of the visualized data, not the number of records. We first describe a design space of scalable visual summaries that use data reduction methods (such as binning aggregation or sampling) to visualize a variety of data types. We then contribute methods for interactive querying (e.g., breaking λ data) among binning plots through a combination of multivariate data tiles and parallel query processing. We implement our techniques in imMens, a browser-based visual analysis system that uses WebGL for data processing and rendering on the GPU. Its benchmarking results show 50 frames per second rendering λ binning among dozens of visualizations, with interactive performance on data sets ranging from thousands to billions of records.

Categories and Subject Descriptors (ccs)

IEEE TRANSACTIONS ON VISUALIZATION AND COMPUTER GRAPHICS, VOL. 20, NO. 1, JANUARY 2013

Gaussian Cubes: Real-Time Modeling for Visual Exploration of Large Multidimensional Datasets

Zhe Wang, Nuno Ferreira, Yuhao Wei, Aarthy Sarkar, Bhaskar, Carlos Scheidegger

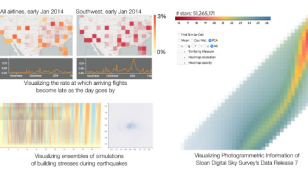


Fig. 1. Some application scenarios for Gaussian Cubes. Top left: screenshot of an interactive section of visual analysis of the Bureau of Transportation Statistics (BTS) on-line performance data, including 140 million light years over a 20-year time span. Gaussian Cubes enable visualizations that describe the slope of the model that describes the light years per day as the day progresses, and these models can be computed over arbitrary subsets of the data at interactive rates. This makes it easy to spot Southeast Airlines alleged practice of scheduling grounding (but not canceling) delayed flights in early 2014. Subsequently, the Department of Transportation issued a large antitrust proceeding (the Boeing-United Jet Survey Data Release 713), which includes 15 million data after data cleaning. Bottom left: Gaussian Cubes used as the backbone data for a large number of earthquake simulations, enabling fast computation of Principal Component Analysis over arbitrary data subsets. See Section 6 for more details.

Abstract—Recently proposed techniques have finally made it possible for analysts to interactively explore very large datasets in real time. However, powerful, the class of analysis these systems enable is somewhat limited: specifically, one can only query data points (such as histograms and heatmaps). In this paper, we contribute Gaussian Cubes, which significantly improves on these techniques by providing interactive multidimensional queries, which include not only the most basic queries and principal components analysis (PCA). The fundamental insight in Gaussian Cubes is that instead of precomputing counts of many data points at each point, Gaussian Cubes precompute the counts of many data points for the respective data subsets. As an example, Gaussian Cubes can fit hundreds of models over millions of data points in well under a second, enabling novel types of visual exploration of such large datasets. We present three case studies that highlight the visualization and analysis capabilities of Gaussian Cubes, using earthquake safety simulations, astronomical catalogs, and transportation statistics. The dataset cube spans around one hundred million elements and 5-6 10 dimensions. We present extensive performance results, a discussion of the limitations in Gaussian Cubes, and future research directions.

Index Terms—Data modeling, dimensionality reduction, interactive visualization, data cubes

1. INTRODUCTION

The fundamental difficulty in visual exploration of large datasets can be summarized by two conflicting requirements. First, exploratory analysis requires a large variety of different queries against the dataset, and these queries are not known before the dataset is processed for visualization. Such a constraint naturally pushes implementations towards especially powerful—but computationally naive—strategies, such as repeated linear scans of the data. Second, user-interaction constraints demand that the quality of the experience is bound by the ease with which analysts can go through an “exploratory hypothesis testing” process, a sequence of iterative queries, leading to detecting the results, examining them, and finally refining some unexpected hypothesis in their mind. This constraint, in turn, pushes implementations

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Wang et al., 2017

DOI: 10.1109/13398

Paragika Conference on Visualization (EuroVis) 2013
J. Heo, C. Lins, and T. Kopsch
(Guest Editors)

Volume 17 (2013), Number 3

Time Lattice: A Data Structure for the Interactive Visual Analysis of Large Time Series

Fabio Miranda¹, Marcos Lago², Harish Dhinasekaran¹, Charles Molyard¹, Justin Salamon¹, Vinodh Lakkemani¹, Juliana Freire¹, Claudio T. Silva²

¹New York University, New York, United States

²Universidade Federal Fluminense, Niterói, Brazil

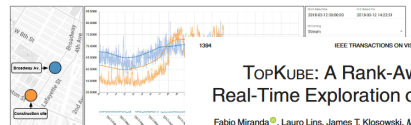


Figure 1: Using Noise Profiler to analyze OLAP queries over a database for arbitrary noise (smooth line), highlighting if noise (blue) is close to a noise node (blueberry, blue) and has a close to a major contribution to the data and has a distinctly higher all data (blueberry line) can be used to get instantaneous information, enforcement means about possible noise code violations such.

Abstract

Advances in technology coupled with the availability of time series from several sources. In order to visually λ various online analytical processing (OLAP) queries λ Effective visual analysis requires these queries to be supported interactive query rates, the exponential data sets. Moreover, none of the recent paper-efficient as whenever there is new data, making them impractical data structure that makes use of the implicit temporal Time Lattice is a subset of a fully materialized cube λ an experimental evaluation which shows that the space compared to the state of the art. In collaboration with λ data structure to design the Noise Profiler λ with them. We demonstrate the utility of Noise Profiler through a λ

1. Introduction

With the massive adoption of the Internet of Things (IoT) we λ accurate parsing from smart home devices and smart cities to λ and healthcare applications, interactive visualization frames are becoming paramount in the exploration and analysis of the

© 2013 IEEE. Authorized Copyright License from IEEE Xplore. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on January 28, 2015 at 10:34:22 UTC from IEEE Xplore. Restrictions apply.

TopKUBE: A Rank-Aware Data Cube for Real-Time Exploration of Spatiotemporal Data

Fabio Miranda¹, Lauro Lins, James T. Klosowski, Member, IEEE, and Claudio T. Silva, Fellow, IEEE

Abstract—From economics to sports to entertainment and social media, ranking objects according to some notion of importance is a fundamental task we humans use of the world to better understand our world. With the ever-increasing amount of user-generated content-based cubes, “what’s trending” is now a commonplace phrase that tries to capture the zeitgeist of the world by ranking the most popular multidimensional trending in a given time and space. However, before we can understand what these trending sets are about the world, we need to be able to more easily create and explore them, given the significant scale of today’s data. In this paper, we describe the conceptual challenges in building a real-time visual exploration tool for finding top-k trending objects, built on the recent work involving in-memory and rank-aware data cubes to propose TopKUBE, a data structure that answers top-k queries up to one order of magnitude faster than the previous state of the art, demonstrates the usefulness of our methods using a set of real-world, publicly available datasets, and provide a new set of benchmarks for other researchers to validate their methods and compare to our own.

Index Terms—Interactive visualization, data cubes, top-K queries, rank merging

1. INTRODUCTION

Ranks and lists play a major role in human context for us to rank everything, from movies to sports teams to the “Top 500” of the world. It is a natural world that is increasingly more our focusing on a subset of objects. There is a way to describe a decade than by ranking songs or movies. One just needs to look at the 100 of any year to gain insight into how the society used to think and behave. High Fidelity [1] person observed with compiling top five lists λ is summarized in the book, the act of a structure to life, by clarifying the point [2]. With the ever-increasing amount of λ content found online, ranks have become our cultural landscape. “What’s trending” is a phrase used to capture the zeitgeist looking at the most popular microblogging λ some way that the most popular songs can describe the zeitgeist of a year or a decade, it can be used to describe the zeitgeist of a day or λ . In addition to the deluge of new user-generated of GPS-enabled devices providing the people creating this content by providing, λ their location when passing a message to Facebook λ picture to Flickr, checking into FourSquare λ

NeuralCubes: Deep Representations for Visual Data Exploration

Zhe Wang, Dept. of Computer Science, University of Arizona, Tucson, AZ, USA, zhw@cs.arizona.edu
Dylan Cushman, Dept. of Computer Science, Tufts University, Medford, MA, USA, dylan.cushman@tufts.edu
Matthew Berger, Dept. of Computer Science, Vanderbilt University, Nashville, TN, USA, matthew.berger@vanderbilt.edu
Joshua A. Levine, Dept. of Computer Science, Tufts University, Medford, MA, USA, josh.levine@tufts.edu
Mingwei Li, Dept. of Computer Science, University of Arizona, Tucson, AZ, USA, mli@cs.arizona.edu
Jixian Li, Dept. of Computer Science, University of Arizona, Tucson, AZ, USA, jixian.li@cs.arizona.edu
Carlos Scheidegger, Dept. of Computer Science, University of Arizona, Tucson, AZ, USA, cscheide@cs.arizona.edu
Remco Chang, Dept. of Computer Science, Tufts University, Medford, MA, USA, remco.c@tufts.edu

Abstract—Visual exploration of large multi-dimensional datasets has been a long-standing problem. In recent years, allowing users to express rich data queries that produce informative visual summaries, all in real time. Techniques based on data cubes are some of the most promising approaches. However, these techniques usually require a large memory footprint for large datasets. To tackle this problem, we present NeuralCubes: neural networks that predict results for aggregate queries, and temporal intervals, and output the result of the query. The learned function serves as a real-time, low-memory approximator for aggregation queries. Our models are small enough to be sent to the client side (as the web browser for a web-based application) for evaluation, enabling data exploration of large datasets without datanetwork connection. We demonstrate the effectiveness of NeuralCubes through extensive experiments: a variety of datasets and discuss how NeuralCubes opens up opportunities for new types of visualization and interaction.

1. INTRODUCTION

Interactive visual exploration is becoming increasingly essential for making sense of large multi-dimensional datasets. It is common for datasets to have billions of data items that contain a variety of attributes of geographic, temporal, and categorical nature. Due to this size and complexity, issuing data queries in real-time is often not feasible as it will result in an unreasonable amount of latency. Instead, efficient data structures are often designed for real-time exploration, in form of making raw database queries. These data structures are pre-computed and optimized around queries frequently used by the visualization, such as performing data summaries [33], [43], ranking [38], and applying multivariate statistics [37]. However, while these data structures are effective, they can still be prohibitively large as data size increases. Worse, when data complexity increases (i.e. in terms of the number of dimensions in the data), the sizes of many of these data structures grow exponentially. As a result, these data structures

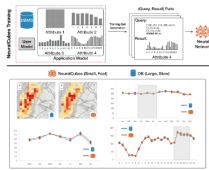


Fig. 1. NeuralCubes frames approximate query processing over a database as a small network, one that takes a query as input, e.g. selection of a temporal interval, and maps it to an aggregation result in output, e.g. a count of the number of records, enabling real-time visual exploration of large datasets. The figure above, the blue lines represent visualizations generated with data from the database and the orange lines are generated using NeuralCubes. Note that the two are nearly indistinguishable, but NeuralCubes uses only a fraction of the memory footprint as the database.

are often stored on a server, and only sub-parts of the data structures are fetched in real-time based on the user’s exploration. In this paper, we introduce NeuralCubes, a technique that approximates queries on multi-dimensional datasets for the support of interactive visual exploration. NeuralCubes is a deep neural network that is compact in storage, can respond to data queries in real time, and is sufficiently accurate for visualization purposes. Thus, unlike existing systems that have high storage requirements and require intricate data structures [33], [34],

Wang et al., 2021

Mosaic: An Architecture for Scalable & Interoperable Data Views

- A middle-tier architecture that sits between interactive components and a backing data source (e.g., DuckDB).

Application:
Visualizations,
interactions, etc.

Database:
Storage + query
execution, indexes,
materialized views

Heer and Moritz, 2024

436

IEEE TRANSACTIONS ON VISUALIZATION AND COMPUTER GRAPHICS, VOL. 30, NO. 1, JANUARY 2024

Mosaic: An Architecture for Scalable & Interoperable Data Views

Jeffrey Heer and Dominik Moritz

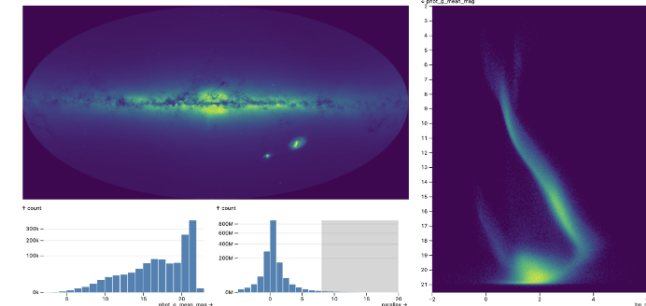


Fig. 1: A Mosaic-based interface for interactive visual exploration of all 1.8 billion stars in the Gaia catalog. A high-resolution density map of the sky reveals our Milky Way and satellite galaxies. Stars with higher parallax values are interactively selected, forming a Hertzprung-Russell diagram of color versus stellar magnitude on the right. Mosaic offloads density and histogram computation to a backing scalable database, and automatically builds optimized data cube indexes to support interactive linked views.

Abstract—Mosaic is an architecture for greater scalability, extensibility, and interoperability of interactive data views. Mosaic decouples data processing from specification logic: clients publish their data needs as declarative queries that are then managed and automatically optimized by a coordinator that proxies access to a scalable data store. Mosaic generalizes Vega-Lite’s selection abstraction to enable rich integration and linking across visualizations and components such as menus, text search, and tables. We demonstrate Mosaic’s expressiveness, extensibility, and interoperability through examples that compose diverse visualization, interaction, and optimization techniques—many constructed using vplot, a grammar of interactive graphics in which graphical marks act as Mosaic clients. To evaluate scalability, we present benchmark studies with order-of-magnitude performance improvements over existing web-based visualization systems—enabling flexible, real-time visual exploration of billion+ record datasets. We conclude by discussing Mosaic’s potential as an open platform that bridges visualization languages, scalable visualization, and interactive data systems more broadly.

Index Terms—Visualization, Interaction, Scalability, Grammar of Graphics, Software Architecture, Databases

1 INTRODUCTION

Though many expressive visualization tools exist, scalability to large datasets and interoperability across tools remain challenging [7]. The visualization community lacks open, standardized tools for integrating visualization specifications with scalable analytic databases. While libraries like D3 [8] embrace Web standards for cross-tool interoperability, higher-level frameworks often make closed-world assumptions, complicating integration with other tools and environments.

As a concrete example, consider the Vega [36] and Vega-Lite [35] ecosystem. By default, data transformations are performed within a JavaScript runtime, limiting scalability due to both data movement and a lack of parallel computing. Meanwhile, other architectural decisions impede extensibility and interoperability. Vega-Lite’s selection

abstraction provides a powerful, concise model for interaction, yet is realized in terms of opaque internal Vega constructs that complicate coordination with external components and environments such as notebooks. As a result, online requests¹ for table views, data-driven input widgets, more interoperable selections, and new mark types (e.g., for raster heatmaps and contour plots in Vega-Lite) have gone unaddressed for years. Other tools face similar limitations [4, 43].

We propose a standardized “middle-tier” architecture that mediates data-driven components and backing data sources. A common layer between databases and components can coordinate linked selections and parameters among views, while providing automatic query optimizations for greater scalability. We focus on the Web browser as the primary site of rendering and interaction, and seek to coordinate diverse components via standard protocols for communicating data needs, dynamic parameters, and linked selection criteria.

We contribute *Mosaic*, an architecture for interoperable, data-driven components—including visualizations, tables, and input widgets—backed by scalable data stores. A key idea of *Mosaic* is to decouple data processing from specification. *Mosaic Clients* communicate their data needs as declarative queries. A central *Coordinator* manages these queries, applies automatic optimizations, and pushes processing to a

• Jeffrey Heer is with University of Washington. E-mail: jheer@uw.edu.
• Dominik Moritz is with Carnegie Mellon University. E-mail: domoritz@cmu.edu.

Manuscript received 31 March 2023; revised 1 July 2023; accepted 8 August 2023.
Date of publication 26 October 2023; date of current version 21 December 2023.
This article has supplementary downloadable material available at <https://doi.org/10.1109/TVCG.2023.3327189>, provided by the authors.
Digital Object Identifier no. 10.1109/TVCG.2023.3327189

¹See github.com/vega/vega/issues/ and github.com/vega/vega-lite/issues/

Mosaic: An Architecture for Scalable & Interoperable Data Views

- A middle-tier architecture that sits between interactive components and a backing data source (e.g., DuckDB).

Application:
Visualizations,
interactions, etc.

**Up to the user to manage linked interaction state,
coordinate multiple visualization components**

Database:
Storage + query
execution, indexes,
materialized views

Heer and Moritz, 2024

436

IEEE TRANSACTIONS ON VISUALIZATION AND COMPUTER GRAPHICS, VOL. 30, NO. 1, JANUARY 2024

Mosaic: An Architecture for Scalable & Interoperable Data Views

Jeffrey Heer and Dominik Moritz

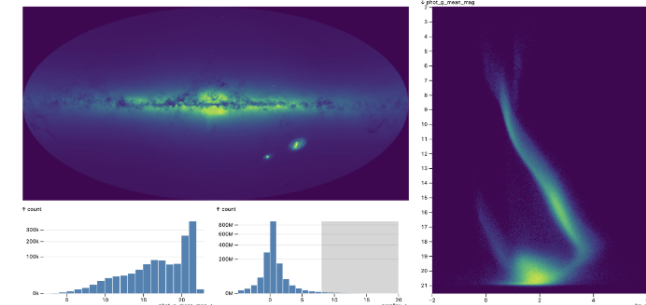


Fig. 1: A Mosaic-based interface for interactive visual exploration of all 1.8 billion stars in the Gaia catalog. A high-resolution density map of the sky reveals our Milky Way and satellite galaxies. Stars with higher parallax values are interactively selected, forming a Hertzprung-Russell diagram of color versus stellar magnitude on the right. Mosaic offloads density and histogram computation to a backing scalable database, and automatically builds optimized data cube indexes to support interactive linked views.

Abstract—Mosaic is an architecture for greater scalability, extensibility, and interoperability of interactive data views. Mosaic decouples data processing from specification logic: clients publish their data needs as declarative queries that are then managed and automatically optimized by a coordinator that proxies access to a scalable data store. Mosaic generalizes Vega-Lite’s selection abstraction to enable rich integration and linking across visualizations and components such as menus, text search, and tables. We demonstrate Mosaic’s expressiveness, extensibility, and interoperability through examples that compose diverse visualization, interaction, and optimization techniques—many constructed using vplot, a grammar of interactive graphics in which graphical marks act as Mosaic clients. To evaluate scalability, we present benchmark studies with order-of-magnitude performance improvements over existing web-based visualization systems—enabling flexible, real-time visual exploration of billion+ record datasets. We conclude by discussing Mosaic’s potential as an open platform that bridges visualization languages, scalable visualization, and interactive data systems more broadly.

Index Terms—Visualization, Interaction, Scalability, Grammar of Graphics, Software Architecture, Databases

1 INTRODUCTION

Though many expressive visualization tools exist, scalability to large datasets and interoperability across tools remain challenging [7]. The visualization community lacks open, standardized tools for integrating visualization specifications with scalable analytic databases. While libraries like D3 [8] embrace Web standards for cross-tool interoperability, higher-level frameworks often make closed-world assumptions, complicating integration with other tools and environments.

As a concrete example, consider the Vega [36] and Vega-Lite [35] ecosystem. By default, data transformations are performed within a JavaScript runtime, limiting scalability due to both data movement and a lack of parallel computing. Meanwhile, other architectural decisions impede extensibility and interoperability. Vega-Lite’s selection

abstraction provides a powerful, concise model for interaction, yet is realized in terms of opaque internal Vega constructs that complicate coordination with external components and environments such as notebooks. As a result, online requests¹ for table views, data-driven input widgets, more interoperable selections, and new mark types (e.g., for raster heatmaps and contour plots in Vega-Lite) have gone unaddressed for years. Other tools face similar limitations [4, 43].

We propose a standardized “middle-tier” architecture that mediates data-driven components and backing data sources. A common layer between databases and components can coordinate linked selections and parameters among views, while providing automatic query optimizations for greater scalability. We focus on the Web browser as the primary site of rendering and interaction, and seek to coordinate diverse components via standard protocols for communicating data needs, dynamic parameters, and linked selection criteria.

We contribute *Mosaic*, an architecture for interoperable, data-driven components—including visualizations, tables, and input widgets—backed by scalable data stores. A key idea of *Mosaic* is to decouple data processing from specification. *Mosaic Clients* communicate their data needs as declarative queries. A central *Coordinator* manages these queries, applies automatic optimizations, and pushes processing to a

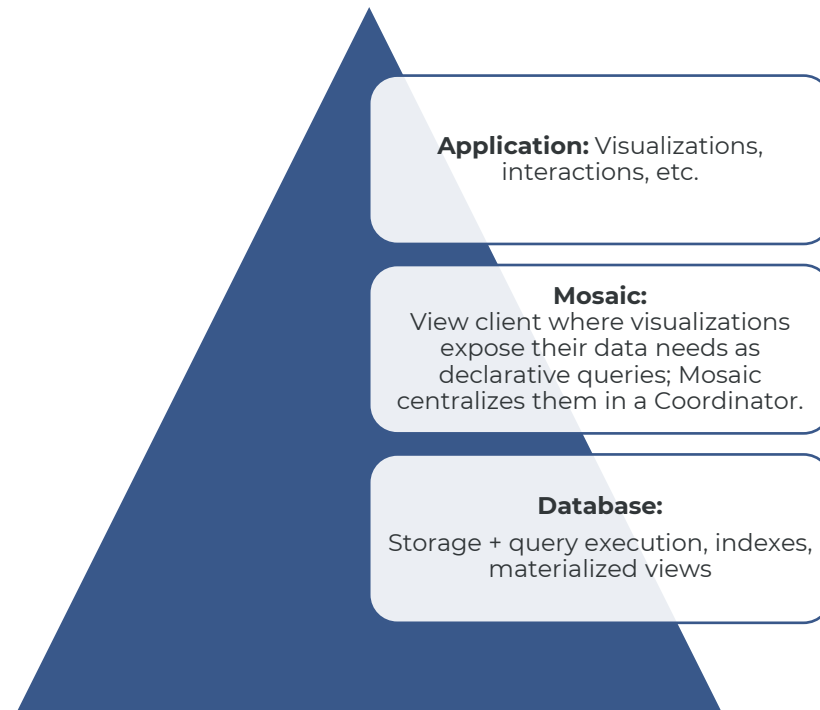
• Jeffrey Heer is with University of Washington. E-mail: jheer@uw.edu.
• Dominik Moritz is with Carnegie Mellon University. E-mail: domoritz@cmu.edu.

Manuscript received 31 March 2023; revised 1 July 2023; accepted 8 August 2023.
Date of publication 26 October 2023; date of current version 21 December 2023.
This article has supplementary downloadable material available at <https://doi.org/10.1109/TVCG.2023.3327189>, provided by the authors.
Digital Object Identifier no. 10.1109/TVCG.2023.3327189

¹See github.com/vega/vega/issues/ and github.com/vega/vega-lite/issues/

Mosaic: An Architecture for Scalable & Interoperable Data Views

- A middle-tier architecture that sits between interactive components and a backing data source (e.g., DuckDB).



Heer and Moritz, 2024

436

IEEE TRANSACTIONS ON VISUALIZATION AND COMPUTER GRAPHICS, VOL. 30, NO. 1, JANUARY 2024

Mosaic: An Architecture for Scalable & Interoperable Data Views

Jeffrey Heer and Dominik Moritz

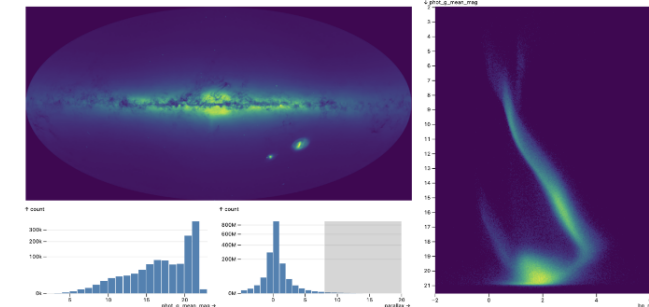


Fig. 1: A Mosaic-based interface for interactive visual exploration of all 1.8 billion stars in the Gaia star catalog. A high-resolution density map of the sky reveals our Milky Way and satellite galaxies. Stars with higher parallax values are interactively selected, forming a Hertzprung-Russell diagram of color versus stellar magnitude on the right. Mosaic offloads density and histogram computation to a backing scalable database, and automatically builds optimized data cube indexes to support interactive linked views.

Abstract—Mosaic is an architecture for greater scalability, extensibility, and interoperability of interactive data views. Mosaic decouples data processing from specification logic: clients publish their data needs as declarative queries that are then managed and automatically optimized by a coordinator that proxies access to a scalable data store. Mosaic generalizes Vega-Lite’s selection abstraction to enable rich integration and linking across visualizations and components such as menus, text search, and tables. We demonstrate Mosaic’s expressiveness, extensibility, and interoperability through examples that compose diverse visualization, interaction, and optimization techniques—many constructed using vplot, a grammar of interactive graphics in which graphical marks act as Mosaic clients. To evaluate scalability, we present benchmark studies with order-of-magnitude performance improvements over existing web-based visualization systems—enabling flexible, real-time visual exploration of billion+ record datasets. We conclude by discussing Mosaic’s potential as an open platform that bridges visualization languages, scalable visualization, and interactive data systems more broadly.

Index Terms—Visualization, Interaction, Scalability, Grammar of Graphics, Software Architecture, Databases

1 INTRODUCTION

Though many expressive visualization tools exist, scalability to large datasets and interoperability across tools remain challenging [7]. The visualization community lacks open, standardized tools for integrating visualization specifications with scalable analytic databases. While libraries like D3 [8] embrace Web standards for cross-tool interoperability, higher-level frameworks often make closed-world assumptions, complicating integration with other tools and environments.

As a concrete example, consider the Vega [36] and Vega-Lite [35] ecosystem. By default, data transformations are performed within a JavaScript runtime, limiting scalability due to both data movement and a lack of parallel computing. Meanwhile, other architectural decisions impede extensibility and interoperability. Vega-Lite’s selection

abstraction provides a powerful, concise model for interaction, yet is realized in terms of opaque internal Vega constructs that complicate coordination with external components and environments such as notebooks. As a result, online requests¹ for table views, data-driven input widgets, more interoperable selections, and new mark types (e.g., for raster heatmaps and contour plots in Vega-Lite) have gone unaddressed for years. Other tools face similar limitations [4, 43].

We propose a standardized “middle-tier” architecture that mediates data-driven components and backing data sources. A common layer between databases and components can coordinate linked selections and parameters among views, while providing automatic query optimizations for greater scalability. We focus on the Web browser as the primary site of rendering and interaction, and seek to coordinate diverse components via standard protocols for communicating data needs, dynamic parameters, and linked selection criteria.

We contribute *Mosaic*, an architecture for interoperable, data-driven components—including visualizations, tables, and input widgets—backed by scalable data stores. A key idea of *Mosaic* is to decouple data processing from specification. *Mosaic Clients* communicate their data needs as declarative queries. A central *Coordinator* manages these queries, applies automatic optimizations, and pushes processing to a

• Jeffrey Heer is with University of Washington. E-mail: jheer@uw.edu.
• Dominik Moritz is with Carnegie Mellon University. E-mail: domoritz@cmu.edu.

Manuscript received 31 March 2023; revised 1 July 2023; accepted 8 August 2023.
Date of publication 26 October 2023; date of current version 21 December 2023.
This article has supplementary downloadable material available at <https://doi.org/10.1109/TVCG.2023.3327189>, provided by the authors.
Digital Object Identifier no. 10.1109/TVCG.2023.3327189

¹See github.com/vega/vega/issues/ and github.com/vega/vega-lite/issues/

Mosaic: An Architecture for Scalable & Interoperable Data Views

- DuckDB will answer queries.
- Mosaic will decide which queries exist, when they run, and how to optimize workloads globally.
 - Query caching
 - Query consolidation
 - Prefetching
 - Automatic data cube indexes

Heer and Moritz, 2024

436

IEEE TRANSACTIONS ON VISUALIZATION AND COMPUTER GRAPHICS, VOL. 30, NO. 1, JANUARY 2024

Mosaic: An Architecture for Scalable & Interoperable Data Views

Jeffrey Heer and Dominik Moritz

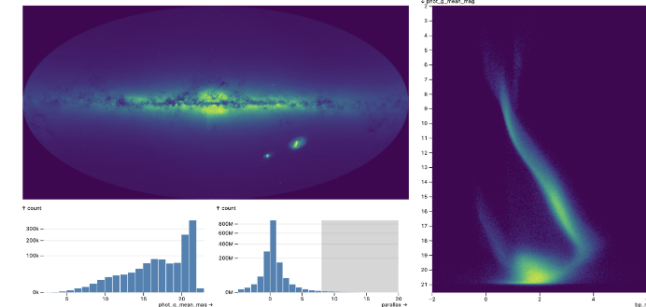


Fig. 1: A Mosaic-based interface for interactive visual exploration of all 1.8 billion stars in the Gaia star catalog. A high-resolution density map of the sky reveals our Milky Way and satellite galaxies. Stars with higher parallax values are interactively selected, forming a Hertzprung-Russell diagram of color versus stellar magnitude on the right. Mosaic offloads density and histogram computation to a backing scalable database, and automatically builds optimized data cube indexes to support interactive linked views.

Abstract—Mosaic is an architecture for greater scalability, extensibility, and interoperability of interactive data views. Mosaic decouples data processing from specification logic: clients publish their data needs as declarative queries that are then managed and automatically optimized by a coordinator that proxies access to a scalable data store. Mosaic generalizes Vega-Lite’s selection abstraction to enable rich integration and linking across visualizations and components such as menus, text search, and tables. We demonstrate Mosaic’s expressiveness, extensibility, and interoperability through examples that compose diverse visualization, interaction, and optimization techniques—many constructed using *vgplot*, a grammar of interactive graphics in which graphical marks act as Mosaic clients. To evaluate scalability, we present benchmark studies with order-of-magnitude performance improvements over existing web-based visualization systems—enabling flexible, real-time visual exploration of billion+ record datasets. We conclude by discussing Mosaic’s potential as an open platform that bridges visualization languages, scalable visualization, and interactive data systems more broadly.

Index Terms—Visualization, Interaction, Scalability, Grammar of Graphics, Software Architecture, Databases

1 INTRODUCTION

Though many expressive visualization tools exist, scalability to large datasets and interoperability across tools remain challenging [7]. The visualization community lacks open, standardized tools for integrating visualization specifications with scalable analytic databases. While libraries like D3 [8] embrace Web standards for cross-tool interoperability, higher-level frameworks often make closed-world assumptions, complicating integration with other tools and environments.

As a concrete example, consider the Vega [36] and Vega-Lite [35] ecosystem. By default, data transformations are performed within a JavaScript runtime, limiting scalability due to both data movement and a lack of parallel computing. Meanwhile, other architectural decisions impede extensibility and interoperability. Vega-Lite’s selection

abstraction provides a powerful, concise model for interaction, yet is realized in terms of opaque internal Vega constructs that complicate coordination with external components and environments such as notebooks. As a result, online requests¹ for table views, data-driven input widgets, more interoperable selections, and new mark types (e.g., for raster heatmaps and contour plots in Vega-Lite) have gone unaddressed for years. Other tools face similar limitations [4, 43].

We propose a standardized “middle-tier” architecture that mediates data-driven components and backing data sources. A common layer between databases and components can coordinate linked selections and parameters among views, while providing automatic query optimizations for greater scalability. We focus on the Web browser as the primary site of rendering and interaction, and seek to coordinate diverse components via standard protocols for communicating data needs, dynamic parameters, and linked selection criteria.

We contribute *Mosaic*, an architecture for interoperable, data-driven components—including visualizations, tables, and input widgets—backed by scalable data stores. A key idea of *Mosaic* is to decouple data processing from specification. *Mosaic Clients* communicate their data needs as declarative queries. A central *Coordinator* manages these queries, applies automatic optimizations, and pushes processing to a

• Jeffrey Heer is with University of Washington. E-mail: jheer@uw.edu.
• Dominik Moritz is with Carnegie Mellon University. E-mail: domoritz@cmu.edu.

Manuscript received 31 March 2023; revised 1 July 2023; accepted 8 August 2023.
Date of publication 26 October 2023; date of current version 21 December 2023.
This article has supplementary downloadable material available at <https://doi.org/10.1109/TVCG.2023.3327189>, provided by the authors.
Digital Object Identifier no. 10.1109/TVCG.2023.3327189

¹See github.com/vega/vega/issues/ and github.com/vega/vega-lite/issues/

Mosaic: An Architecture for Scalable & Interoperable Data Views

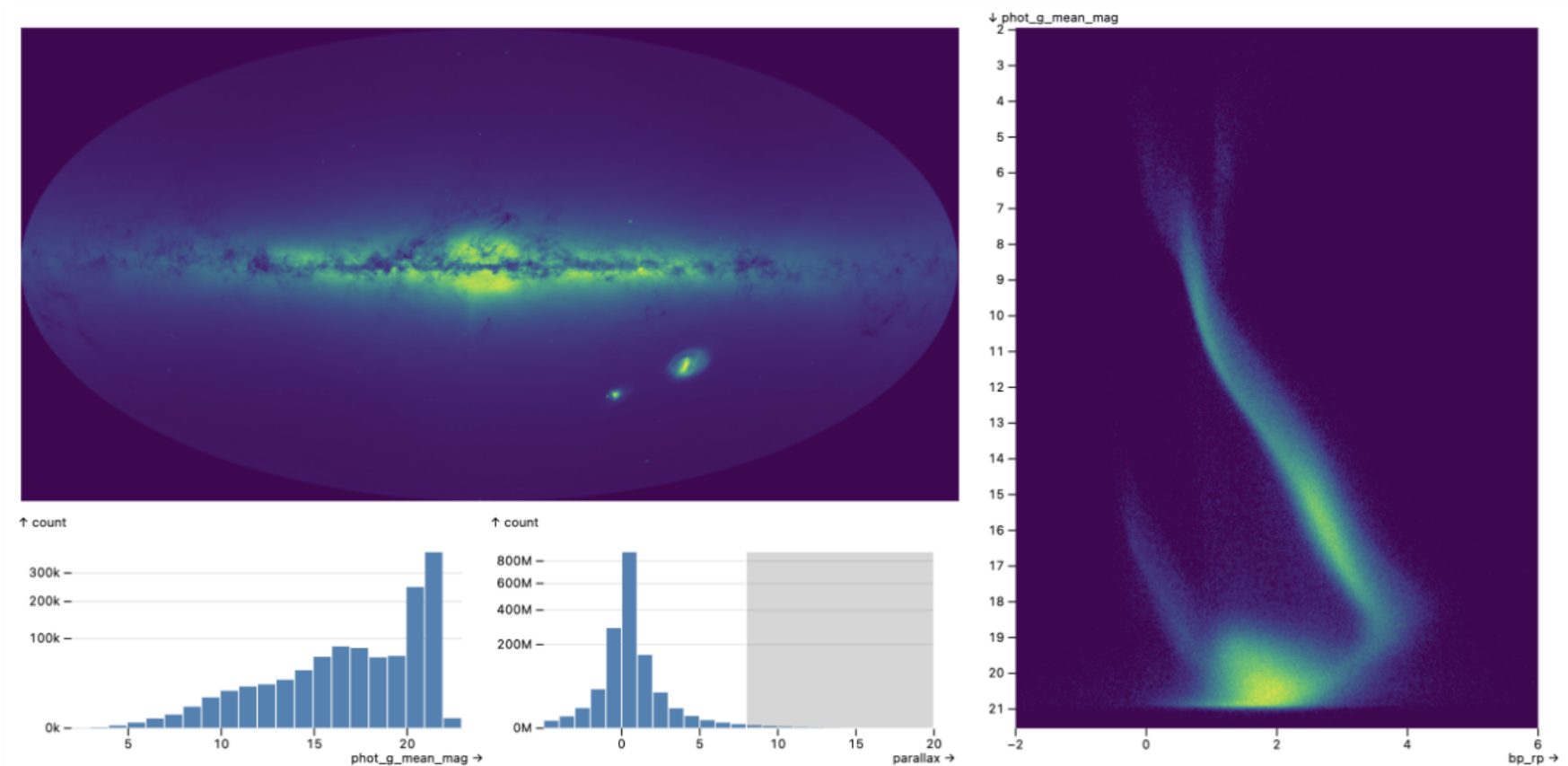


Fig. 1: A Mosaic-based interface for interactive visual exploration of all 1.8 billion stars in the Gaia star catalog. A high-resolution density map of the sky reveals our Milky Way and satellite galaxies. Stars with higher parallax values are interactively selected, forming a [Hertzsprung-Russell diagram](#) of color versus stellar magnitude on the right. Mosaic offloads density and histogram computation to a backing scalable database, and automatically builds optimized data cube indexes to support interactive linked views.

Mosaic: An Architecture for Scalable & Interoperable Data Views

- Query consolidation:
 - When several views request data around the same time, the Coordinator:
 1. Waits one animation frame;
 2. Collects incomings queries; and
 3. Merges those that hit the same table and the same GROUP BY dimensions into a single query.

View 1

```
SELECT month, AVG(price) AS avg_price  
FROM sales  
GROUP BY month;
```

View 2

```
SELECT month, SUM(price) AS total_sales  
FROM sales  
GROUP BY month;
```

Mosaic: An Architecture for Scalable & Interoperable Data Views

- Query consolidation:
 - When several views request data around the same time, the Coordinator:
 1. Waits one animation frame;
 2. Collects incoming queries; and
 3. Merges those that hit the same table and the same GROUP BY dimensions into a single query.

View 1

```
SELECT month, AVG(price) AS avg_price  
FROM sales  
GROUP BY month;
```

View 2

```
SELECT month, SUM(price) AS total_sales  
FROM sales  
GROUP BY month;
```

```
SELECT month, AVG(price) AS avg_price, SUM(price) AS total_sales  
FROM sales  
GROUP BY month;
```

(month, avg_price)

View 1

(month, total_sales)

View 2

Mosaic: An Architecture for Scalable & Interoperable Data Views

- Data cube indexing:
 - Optimizes filtering of aggregated data, building indexes as small data cubes.
 - Without an index, every brush triggers: **filter raw rows → re-aggregate → redraw**
 - With a cube index, each brush triggers: **query a pre-aggregated table → sum a few bins → redraw**
 - Build once, reuse many times: data cubes are built based on actual workload.
 - Index size bounded by resolution, not by the number of data rows.
 - Key point: Mosaic doesn't compute the data cube of the entire dataset, but rather the smallest cube it needs for views at a bounded screen resolution.

Mosaic: An Architecture for Scalable & Interoperable Data Views

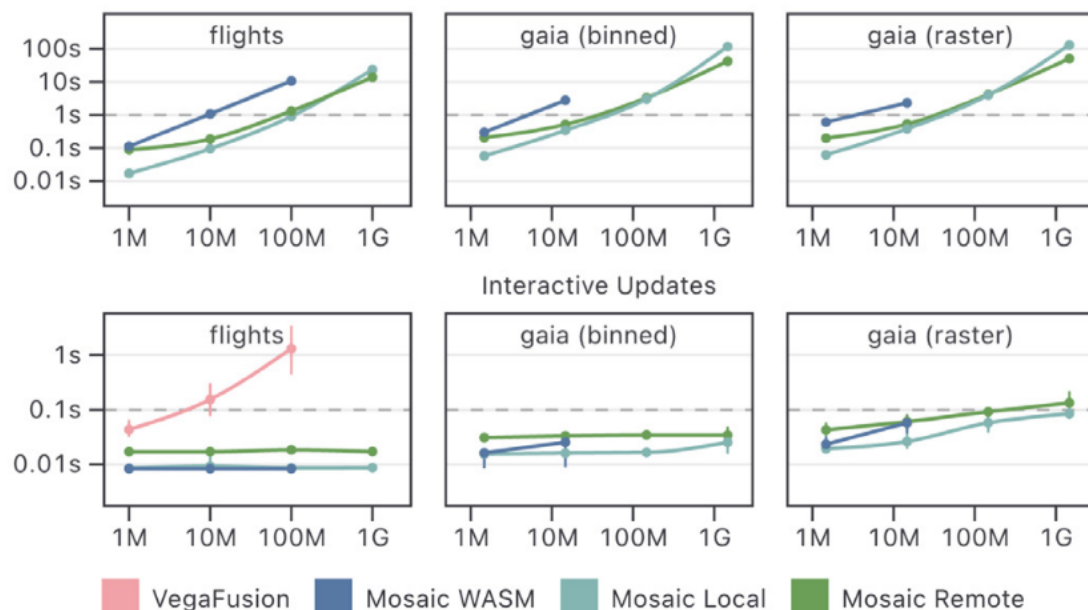


Fig. 17: Index construction (top) and interactive update performance (bottom). Median times and interquartile ranges are shown. Mosaic server configurations maintain interactive update rates at high data volumes, though start to degrade as indexes for raster data become denser.

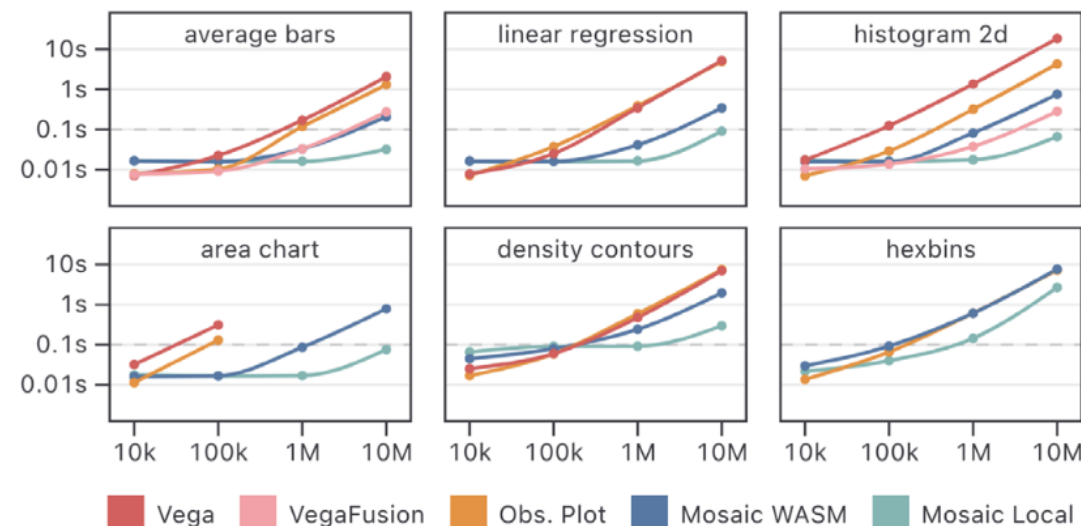


Fig. 16: Initial rendering performance. Median times are shown, interquartile ranges are smaller than the plotted dots. Mosaic provides order-of-magnitude performance improvements over Vega or Observable Plot for a range of visualizations. VegaFusion only optimizes the *average bars* and *2D histogram* conditions. Mosaic WASM does not scale as well as a local server, in part due to the lack of parallel processing.

Mosaic Selections: Managing and Optimizing User Selections for Scalable Data Visualization Systems

Heer et al., 2025

- Focuses on **linked interactions**.
- Interactive systems may have many *dependent* views to update.
- Mosaic Selections make selections explicit as structured objects:
 - Where did the selection come from?
 - Which views should it apply to?
- Formal structure allows for automatic optimizations.

Mosaic Selections: Managing and Optimizing User Selections for Scalable Data Visualization Systems

Jeffrey Heer, Dominik Moritz, Ron Pechuk

Abstract—Though powerful tools for analysis and communication, interactive visualizations often fail to support real-time interaction with large datasets with millions or more records. To highlight and filter data, users indicate values or intervals of interest. Such selections may span multiple components, combine in complex ways, and require optimizations to ensure low-latency updates. We describe Mosaic Selections, a model for representing, managing, and optimizing user selections, in which one or more filter predicates are added to queries that request data for visualizations and input widgets. By analyzing both queries and selection predicates, Mosaic Selections enable automatic optimizations, including pre-aggregating data to rapidly compute selection updates. We contribute a formal description of our selection model and optimization methods, and their implementation in the open-source Mosaic architecture. Benchmark results demonstrate orders-of-magnitude latency improvements for selection-based optimizations over unoptimized queries and existing optimizers for the Vega language. The Mosaic Selection model provides infrastructure for flexible, interoperable filtering across multiple visualizations, alongside automatic optimizations to scale to millions and even billions of records.

Index Terms—scalable visualization, interactive selection, multiple coordinated views, brushing and linking, user interfaces

1 INTRODUCTION

To support effective analysis, interactive data systems should respond to user input at “rates resonant with the pace of human thought” [25, 34], as “milliseconds matter” [18] for user interface latency. To ensure perception of continuity and causality, foundational texts in Human-Computer Interaction [8] draw upon prior psychological studies to advocate interactive response rates under 100ms. Experiments [34, 59] find that adding 500ms of latency to interactive visualizations can alter analysts’ strategies and degrade performance, leading to reduced exploration and hypothesis generation. However, not all interactive operations are equal, as the degree of latency sensitivity may vary [3, 34]. User performance may be robust to a one second delay when creating a new chart, whereas the same latency for navigation operations such as panning and zooming can render an application unusable. In particular, updates for *linked selections*—also known as *brushing and linking* [5]—have been shown to be especially latency sensitive [34].

Though there are many expressive tools for visualization and data applications, most fail to scale to large datasets while preserving low-latency interactions [4, 58]. While researchers have developed methods for low-latency updates in response to dynamic user selections [2, 33, 35, 37, 41, 50, 51], most support only a limited subset of visualization types (such as scatter plots or time series) and interactions (such as panning and zooming). Moreover, many optimizations are not designed to work together in a single system, and require manual tuning or precomputation. Instead, we contend that interface designers and data analysts should be able to specify desired views and interactions at a high-level, and then benefit from automatic optimizations that, when possible, provide low-latency interaction (≤ 100 ms) over backing data volumes with millions or even billions of records.

We contribute a **formal model of user selections** in terms of *client views* that consume data, *interactors* that generate predicate clauses to select data subsets, and *selections* that manage clause sets to resolve view-specific filters. The selection model translates common operations performed in data analysis interfaces (using inputs such as menus, sliders, table views, and interactive visualizations) to filtering predicates

applicable in database queries, and enables coordinated updates across multiple interface components. Unlike many prior formulations [10, 23, 46], our model flexibly supports *cross-filtering* configurations in which selections may apply to some interface views, but not others.

As datasets grow to millions or more rows, rendering all records becomes infeasible due to both perceptual and computational constraints [35]. Instead, it is common to bin, group, and aggregate data to achieve more scalable displays, ranging from basic histograms to high-resolution raster (pixel grid) views. Interactive updates must bin, filter and re-aggregate data, leading to prohibitively high latency if performed against a full dataset upon every update. To speed up these queries, the **selection model automatically optimizes updates using materialized views of pre-aggregated data**.¹ By analyzing client queries and a current active selection clause, the model determines appropriate dimensions and pre-aggregated measures to support efficient interactive updates. Querying the materialized views can make queries multiple orders of magnitude faster, enabling low-latency updates.

General multi-query optimization requires queries ahead of time and is NP-hard [29]. In contrast, queries for interactive visualization applications are predictable [35, 41]. The selection model leverages an application’s structure to make multi-query optimization and answering queries using views [20] tractable. We detail a materialized view construction and query approach, which unlike prior methods [35, 41] supports aggregate operations beyond COUNT and SUM, and enables in-database computation and reuse across sessions and users.

We implemented the selection model and optimizations in Mosaic [24], an open-source architecture for linking databases and interactive views. By default Mosaic leverages DuckDB [44] for flexible deployment in web browsers, Jupyter notebooks, or using database servers. An earlier paper [24] presents the overall Mosaic architecture, describes included visualization and input widget components, and briefly introduces the selection model. We extend that work with a formal selection model, implementation details, and pre-aggregation optimizations with expanded aggregate function support. We also present performance benchmark results demonstrating significant improvements over naïve methods and existing systems, enabling a wider array of real-time interactions with large datasets.

In sum, this paper contributes (1) a formal model of user selections that bridges user interactions and backing databases; (2) optimizations that create and query pre-aggregated materialized views—applied *automatically* based on query and selection clause analysis—to accelerate aggregate queries for rapid interactive updates; and (3) performance benchmarks showing that Mosaic selections can reduce interactive latency by multiple orders of magnitude, outperforming existing systems.

¹A materialized view stores the result of a database query by name.

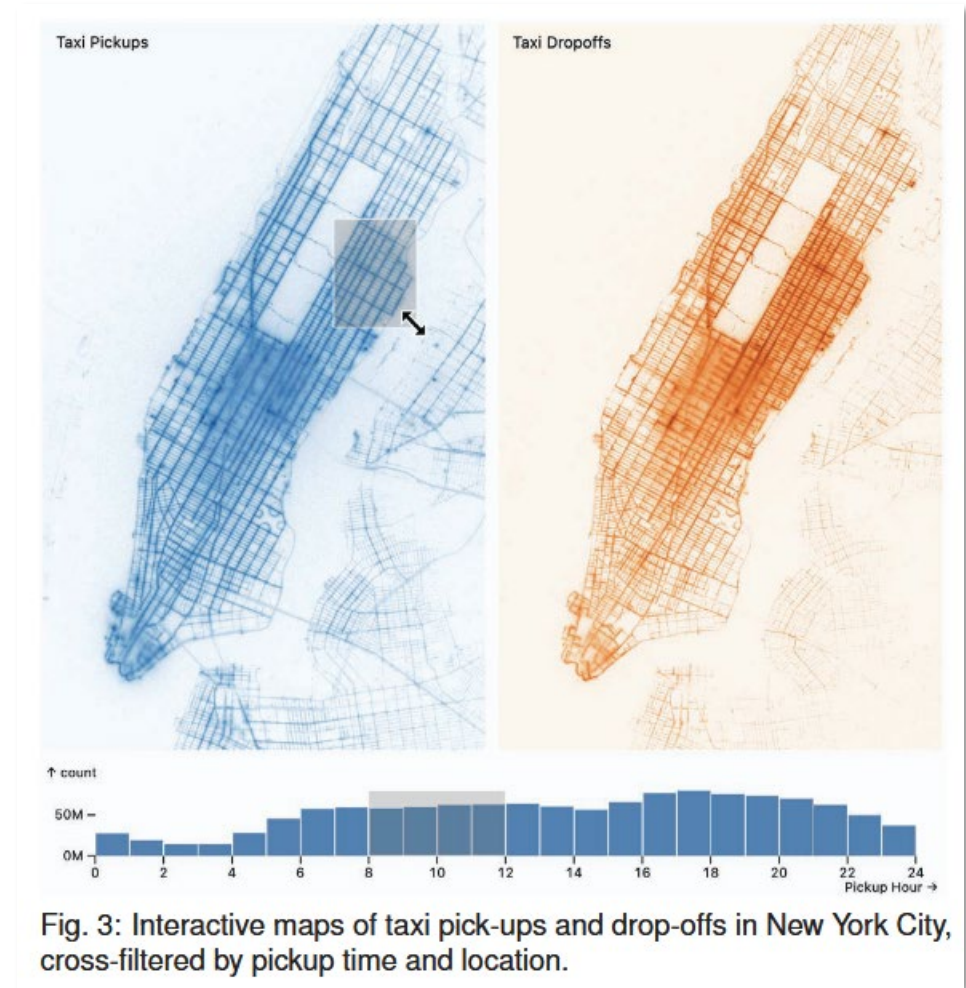
• Jeffrey Heer is with University of Washington. E-mail: jheer@uw.edu.
• Dominik Moritz is with Carnegie Mellon University. E-mail: domoritz@cmu.edu.

• Ron Pechuk is with University of Washington. E-mail: rpechuk@uw.edu.

Manuscript received xx.xxx.201x; accepted xx.xxx.201x. Date of Publication xx.xxx.201x; date of current version xx.xxx.201x. For information on obtaining reprints of this article, please send e-mail to: reprints@ieee.org. Digital Object Identifier: xx.xxxx/TVCG.201x.xxxxxx

Mosaic Selections: Managing and Optimizing User Selections for Scalable Data Visualization Systems

- Focuses on **linked interactions**.
- Interactive systems may have many *dependent* views to update.
- Mosaic Selections make selections explicit as structured objects:
 - Where did the selection come from?
 - Which views should it apply to?
- Formal structure allows for automatic optimizations.



Mosaic Selections: Managing and Optimizing User Selections for Scalable Data Visualization Systems

- Expanded aggregate function support, with std. deviation, covariance, correlation.
- To automatically pre-aggregate data:

1. check if a query q_v filtered by active clause c_a performs aggregation amenable to optimization,
2. determine dimension columns by extracting any groupby values referenced in q_v , as well as dimensions for the possible values of the active clause c_a , and
3. create a materialized view with measure columns containing sufficient statistics for all aggregate query results.

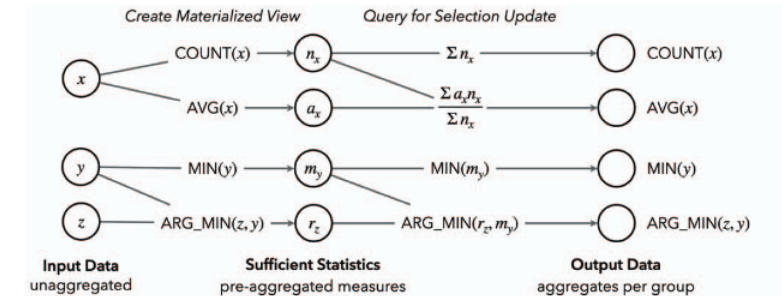


Fig. 8: Pre-aggregation and querying for univariate measures. Each sufficient statistic is included as a column in a materialized view, alongside grouping dimensions.

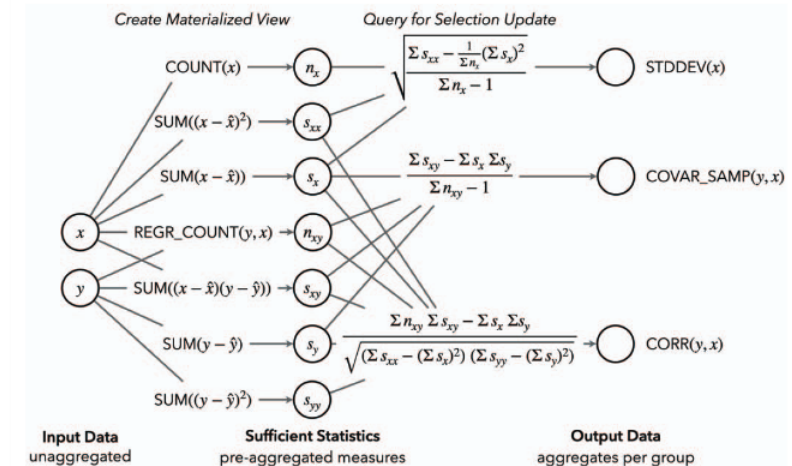


Fig. 9: Pre-aggregation and querying for standard deviation and bivariate measures. Each sufficient statistic is included as a column in a materialized view, alongside grouping dimensions. The symbol $\hat{x}$ indicates the average value of x across the full dataset; it is included to mean-center the data to prevent floating point error.

Mosaic Selections: Managing and Optimizing User Selections for Scalable Data Visualization Systems

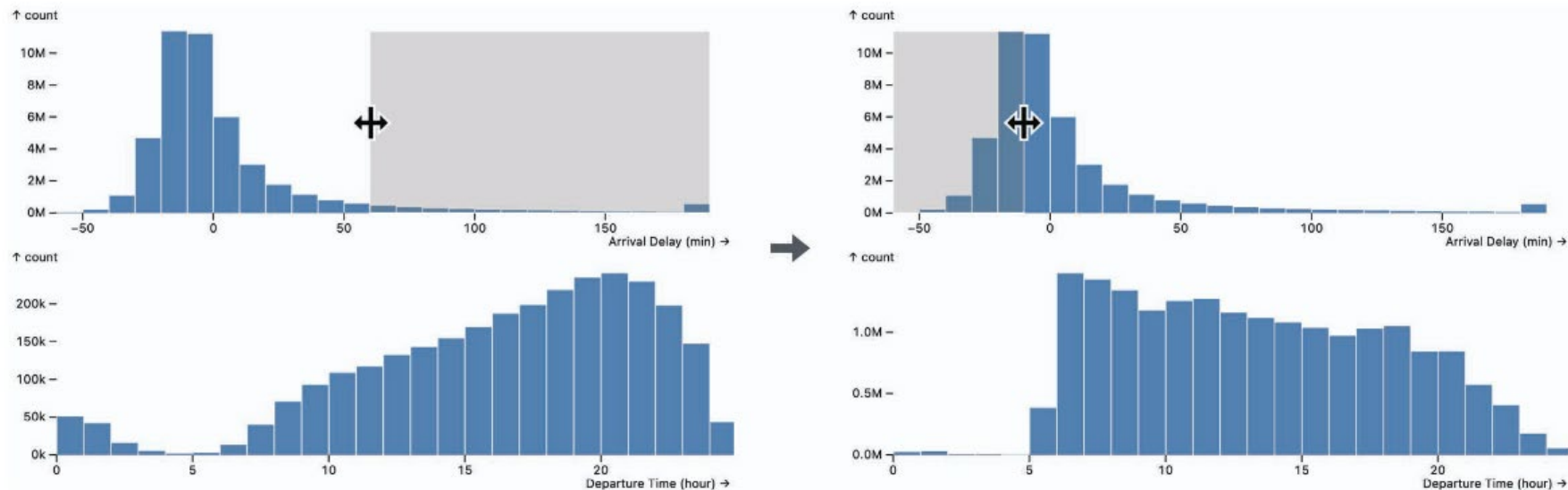


Fig. 1: Histograms of flight arrival data, interactively filtered by arrival delay intervals.

```
SELECT 24 * FLOOR(time / 24) AS x,
COUNT(*) AS y,
FLOOR($pixels * (delay - $min) / ($max - $min)) AS active
FROM flights
WHERE delay BETWEEN $delay_i AND $delay_j
GROUP BY x, active
```

1. check if a query q_v filtered by active clause c_a performs aggregation amenable to optimization,
2. determine dimension columns by extracting any groupby values referenced in q_v , as well as dimensions for the possible values of the active clause c_a , and
3. create a materialized view with measure columns containing sufficient statistics for all aggregate query results.

Mosaic Selections: Managing and Optimizing User Selections for Scalable Data Visualization Systems

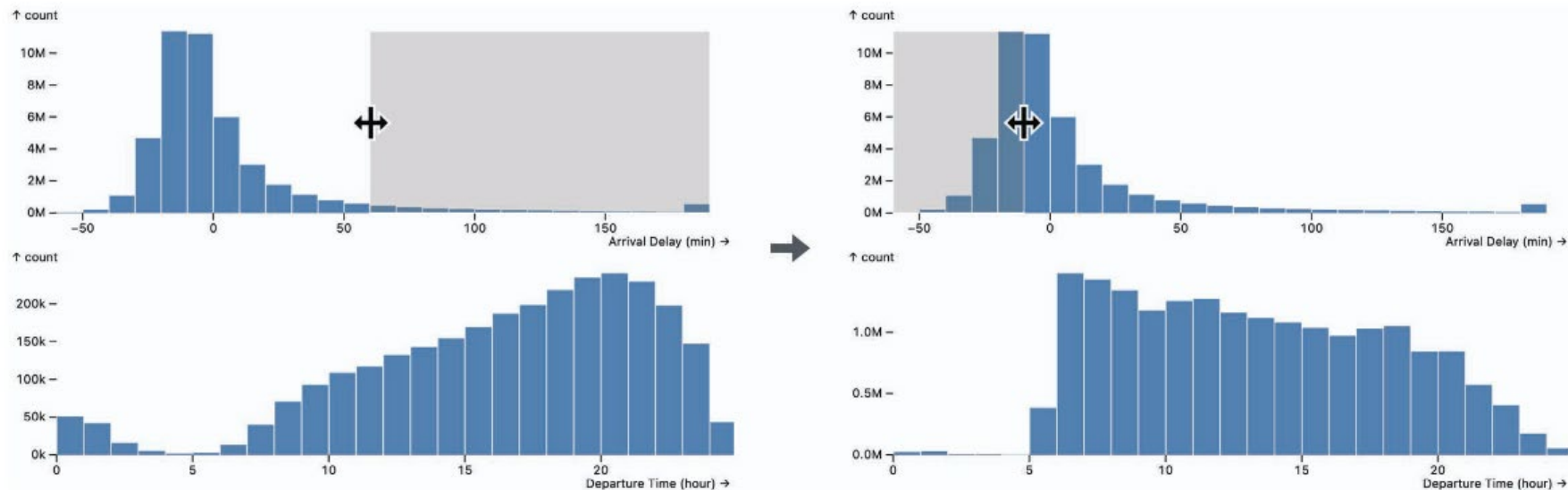


Fig. 1: Histograms of flight arrival data, interactively filtered by arrival delay intervals.

```
SELECT 24 * FLOOR(time / 24) AS x,
COUNT(*) AS y,
FLOOR($pixels * (delay - $min) / ($max - $min)) AS active
FROM flights
WHERE delay BETWEEN $delay_i AND $delay_j
GROUP BY x, active
```

1. check if a query q_v filtered by active clause c_a performs aggregation amenable to optimization,
2. determine dimension columns by extracting any groupby values referenced in q_v , as well as dimensions for the possible values of the active clause c_a , and
3. create a materialized view with measure columns containing sufficient statistics for all aggregate query results.

Mosaic Selections: Managing and Optimizing User Selections for Scalable Data Visualization Systems

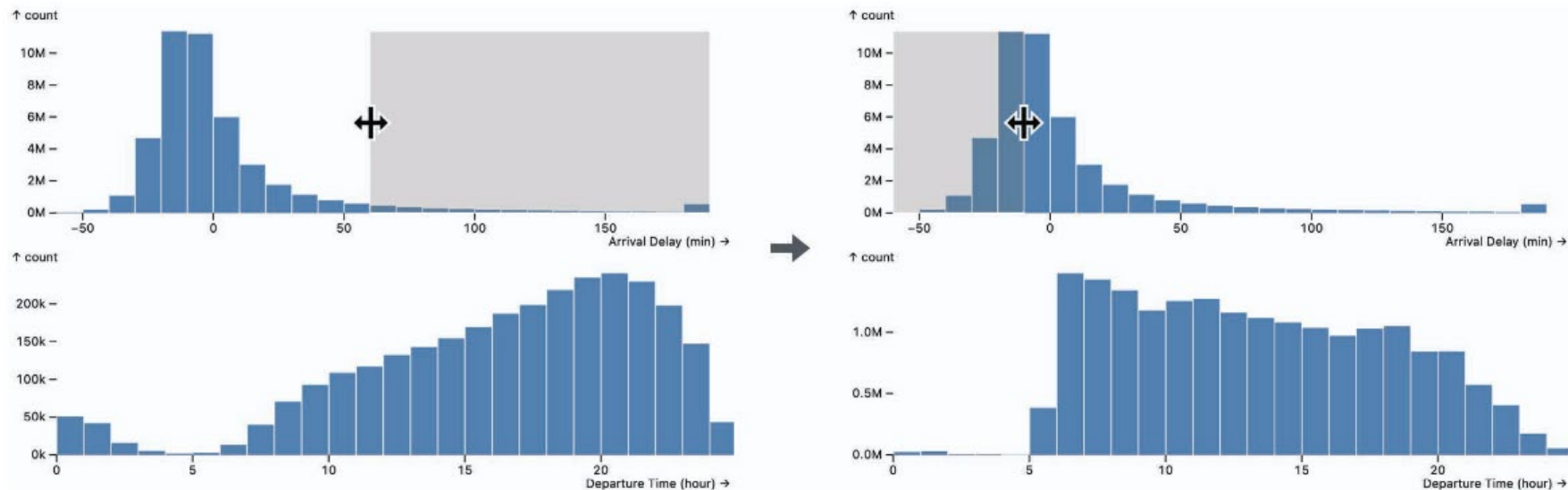


Fig. 1: Histograms of flight arrival data, interactively filtered by arrival delay intervals.

```
SELECT 24 * FLOOR(time / 24) AS x,
COUNT(*) AS y,
FLOOR($pixels * (delay - $min) / ($max - $min)) AS active
FROM flights
WHERE delay BETWEEN $delay_i AND $delay_j
GROUP BY x, active
```

1. check if a query q_v filtered by active clause c_a performs aggregation amenable to optimization,
2. **determine dimension columns by extracting any groupby values referenced in q_v , as well as dimensions for the possible values of the active clause c_a , and**
3. create a materialized view with measure columns containing sufficient statistics for all aggregate query results.

Mosaic Selections: Managing and Optimizing User Selections for Scalable Data Visualization Systems

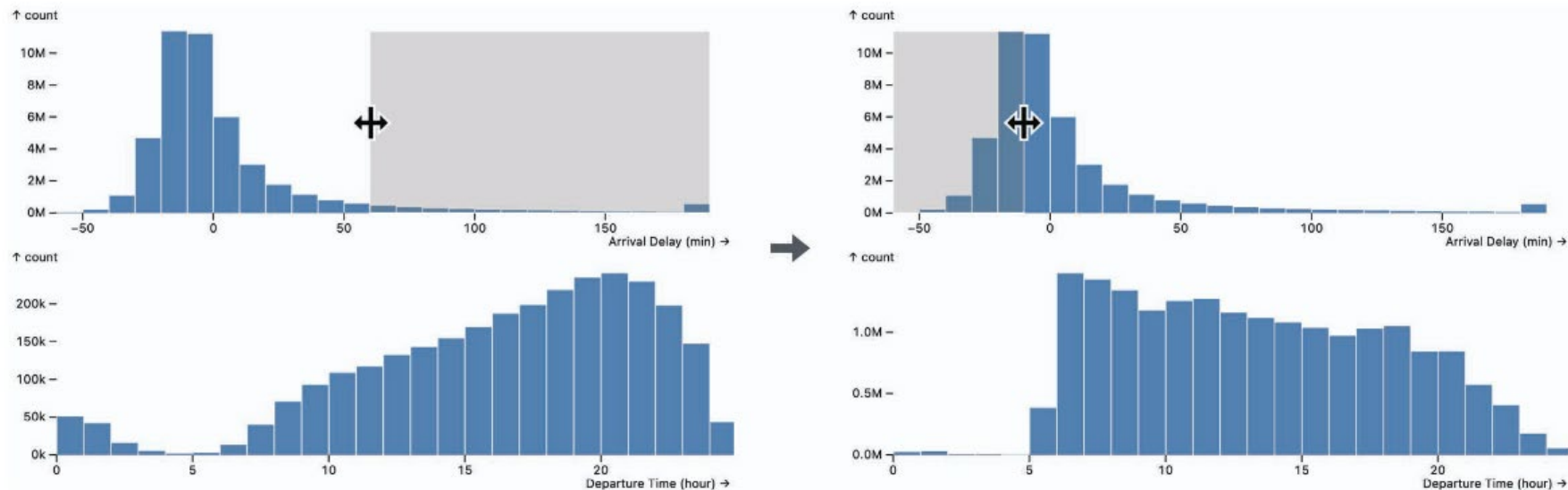


Fig. 1: Histograms of flight arrival data, interactively filtered by arrival delay intervals.

```
SELECT 24 * FLOOR(time / 24) AS x,
COUNT(*) AS y,
FLOOR($pixels * (delay - $min) / ($max - $min)) AS active
FROM flights
WHERE delay BETWEEN $delay_i AND $delay_j
GROUP BY x, active
```

1. check if a query q_v filtered by active clause c_a performs aggregation amenable to optimization,
2. determine dimension columns by extracting any groupby values referenced in q_v , as well as dimensions for the possible values of the active clause c_a , and
- 3. create a materialized view with measure columns containing sufficient statistics for all aggregate query results.**